

Activity 6 - Load inventory JSON with Cargo dependencies

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K2/A2 K4/A4

Specification

Item { sku, name, qty: u32, unit_cents: u64 } derives Serialize and Deserialize. load_items(&str) -> Result<Vec<Item>, serde_json::Error>; to_json(&[Item]) -> Result<String, _>; stock_value(&[Item]) -> Option<u64>.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain Cargo dependencies, Cargo.lock, parsing structured text and what serde automates to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is samples/01_manual_parsing.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on Cargo dependencies, Cargo.lock, parsing structured text and what serde automates with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles packages, dependencies and data formats with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: Item { sku, name, qty: u32, unit_cents: u64 } derives Serialize and Deserialize. load_items(&str) -> Result<Vec<Item>, serde_json::Error>; to_json(&[Item]) -> Result<String, _>; stock_value(&[Item]) -> Option<u64>. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Load inventory JSON with Cargo dependencies" in starter/src so that the supplied tests pass. Use serde derive only; do not hand-write JSON parsing or add other crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_manual_parsing.rs

Run: rustc --edition 2021 01_manual_parsing.rs && ./01_manual_parsing

// Sample: parsing "sku,qty" by hand - the work serde automates for JSON.

```
struct Item {
    sku: String,
    qty: u32,
}

fn parse(line: &str) -> Result<Item, String> {
    let (sku, qty) = line.split_once(',').ok_or("expected sku,qty");
    let qty = qty.trim().parse:::<u32>().map_err(|e| format!("qty: {e}"))?;
    Ok(Item {
        sku: sku.trim().to_string(),
        qty,
    })
}

fn main() {
    for line in ["ABC-0001, 40", "ABC-0002, -1", "no comma"] {
        match parse(line) {
            Ok(item) => {
                println!("{line:?} -> sku={item.sku} qty={item.qty}")
            }
            Err(error) => println!("{line:?} -> error: {error}"),
        }
    }
}
```

Expected output:

```
"ABC-0001, 40" -> sku=ABC-0001 qty=40
"ABC-0002, -1" -> error: qty: invalid digit found in string
"no comma" -> error: expected sku,qty
```

02_serialize_by_hand.rs

Run: rustc --edition 2021 02_serialize_by_hand.rs && ./02_serialize_by_hand

// Sample: writing JSON by hand with Display - fragile, which is why we use serde.
use std::fmt;

```
struct Item {
    sku: String,
    name: String,
    qty: u32,
    unit_cents: u64,
}

impl fmt::Display for Item {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            r#"{"sku": "{}", "name": "{}", "qty": {}, "unit_cents": {}}"#,
            self.sku, self.name, self.qty, self.unit_cents
        )
    }
}
```

```

}

fn main() {
    let item = Item {
        sku: "ABC-0001".into(),
        name: "USB-C cable".into(),
        qty: 40,
        unit_cents: 899,
    };
    println!("{item}");
    println!("Hand-written JSON breaks if a name contains a quote; serde escapes it for you.");
}

```

Expected output:

```

{"sku":"ABC-0001","name":"USB-C cable","qty":40,"unit_cents":899}
Hand-written JSON breaks if a name contains a quote; serde escapes it for you.

```

solution/examples/json_demo.rs (uses serde)

Run: `cd solution && cargo run --example json_demo`

```

//! Run from solution/: cargo run --example json_demo
use activity06::{load_items, stock_value, to_json};

```

```

fn main() {
    let json = r#"["sku":"ABC-0001","name":"USB-C cable","qty":40,"unit_cents":899}]"#;
    let items = load_items(json).expect("valid JSON");
    println!("loaded {} item(s); value = {:?} cents", items.len(), stock_value(&items));
    println!("{}", to_json(&items).expect("serialisable"));
    let bad = r#"["sku":"ABC-0002","name":"Hub","qty":-1,"unit_cents":10}]"#;
    println!("negative qty -> {}", load_items(bad).unwrap_err());
}

```

First line of output: loaded 1 item(s); value = Some(35960) cents

Learn more

Rust Book: Hello, Cargo!: <https://doc.rust-lang.org/book/ch01-03-hello-cargo.html>

Serde: overview and derive: <https://serde.rs/>

Programiz: Cargo: <https://www.programiz.com/rust/cargo>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>